



CHRIST
(DEEMED TO BE UNIVERSITY)
DELHI - NCR, INDIA

NLDL Programming

MCA-575

Assignment – 01

BY

HIMANSHU HEDA (24225013)

SUBMITTED TO

Dr. Manjula Shannhog

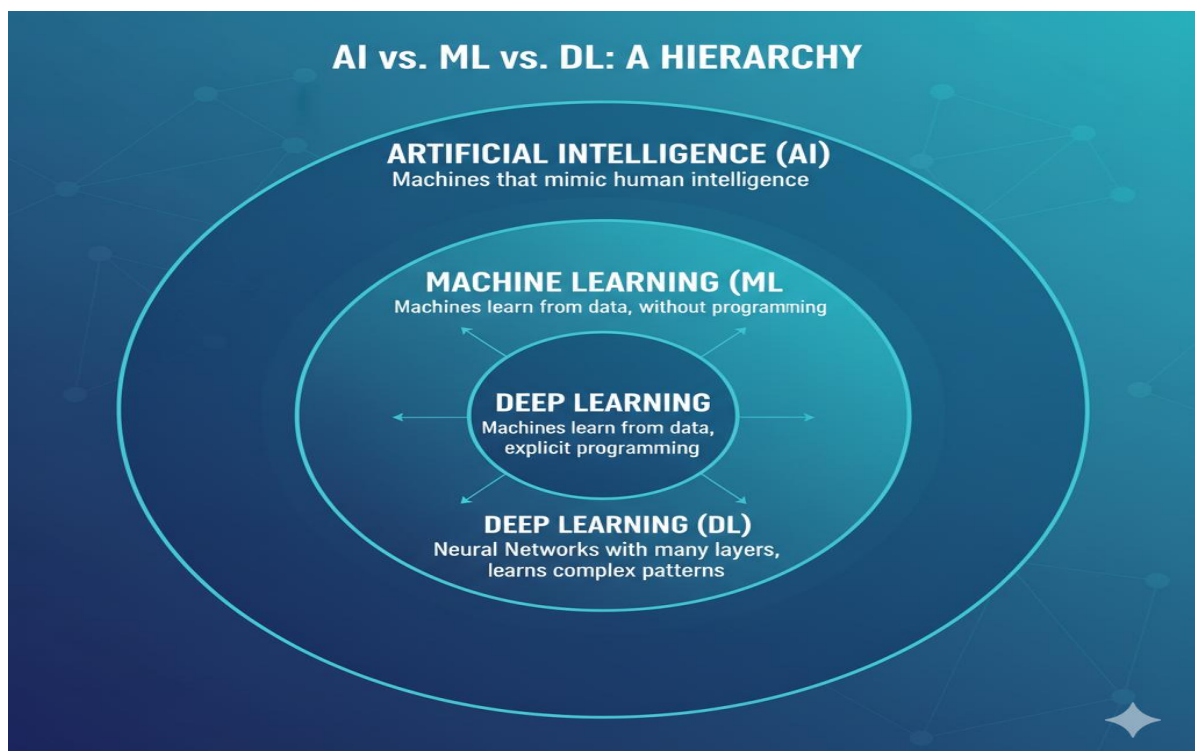
SCHOOL OF SCIENCES

2025-26

1. What is the Difference between AI, ML, and DL?

These three terms are often used interchangeably, but they represent a hierarchy of concepts within computer science.

- **Artificial Intelligence (AI):** This is the broadest concept. AI refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. The goal of AI is to create intelligent systems that can reason, learn, solve problems, perceive, and understand language. Think of it as the overarching field aiming to make machines "smart."
- **Machine Learning (ML):** This is a subset of AI. ML is about enabling machines to learn from data without being explicitly programmed for every task. Instead of writing rules for every possible scenario, you feed an ML model a lot of data, and it learns patterns and relationships from that data to make predictions or decisions. For example, an ML algorithm can learn to distinguish between spam and non-spam emails by analyzing a large dataset of emails.
- **Deep Learning (DL):** This is a specialized subset of Machine Learning. Deep Learning uses artificial neural networks with multiple layers (hence "deep") to learn complex patterns from data. These networks are inspired by the structure and function of the human brain. DL has been particularly successful in tasks like image recognition, natural language processing, and speech recognition, where it can automatically discover features from raw data that traditional ML algorithms would require manual feature engineering for.



2. What is the Difference between Biological Neuron and Artificial Neuron?

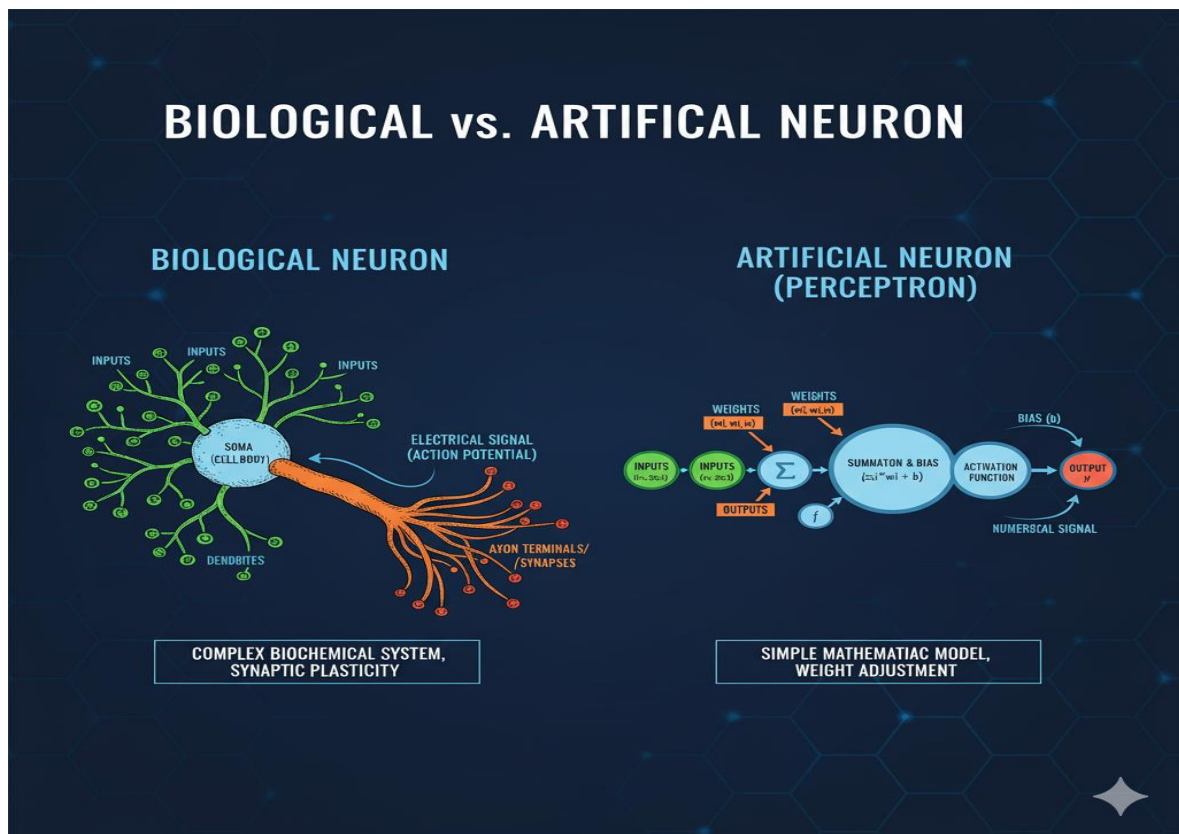
The artificial neuron is a simplified mathematical model inspired by the biological neuron. While they share fundamental principles, there are significant differences.

Biological Neuron: A biological neuron is the fundamental unit of the nervous system, responsible for transmitting information throughout the body and brain.

- **Structure:**
 - **Dendrites:** Receive electrical and chemical signals from other neurons.
 - **Soma (Cell Body):** Integrates the incoming signals.
 - **Axon:** Transmits the output signal to other neurons, muscles, or glands.
 - **Axon Terminals/Synapses:** Where the electrical signal is converted into a chemical signal (neurotransmitters) to be passed to the next neuron.
- **Functionality:**
 - Receives numerous inputs, some excitatory (encouraging firing) and some inhibitory (discouraging firing).
 - If the sum of these inputs reaches a certain threshold, the neuron "fires" (generates an action potential), sending an electrical signal down its axon.
 - Learning occurs through changes in the strength of synaptic connections (synaptic plasticity).
- **Complexity:** Extremely complex, involving a vast array of biochemical processes, ion channels, and neurotransmitter systems.

Artificial Neuron (Perceptron): An artificial neuron, often referred to as a perceptron, is a mathematical function that models the behavior of a biological neuron in a highly simplified way.

- **Structure:**
 - **Inputs ($x_1, x_2, \dots, x_n$):** Represent signals from other neurons. Each input has a weight associated with it.
 - **Weights ($w_1, w_2, \dots, w_n$):** Numerical values that determine the strength or importance of each input.
 - **Summation Function:** Calculates the weighted sum of the inputs ($\sum x_i * w_i$).
 - **Bias (b):** An additional input with a constant value (usually 1) and its own weight, allowing the activation function to be shifted.
 - **Activation Function (f):** Applies a non-linear transformation to the weighted sum (plus bias) to produce an output. Common activation functions include step, sigmoid, ReLU, etc.
 - **Output (y):** The result of the activation function.
- **Functionality:**
 - Receives numerical inputs.
 - Calculates a weighted sum of these inputs.
 - Applies an activation function to the sum.
 - Produces a single numerical output.
 - Learning occurs by adjusting the weights and bias based on the difference between the predicted output and the actual target output.
- **Complexity:** A relatively simple mathematical model, abstracting away most of the biological complexity.



3. What is Perception. Prepare the Structure, Limitation of it

Perceptron: A perceptron is the simplest form of an artificial neural network, representing a single-layer feed-forward network. It was developed by Frank Rosenblatt in 1957. It's a binary linear classifier, meaning it can classify inputs into one of two categories based on a linear decision boundary.

Structure of a Single Perceptron:

1. Input Layer:

- **Inputs ($x_1, x_2, \dots, x_n$):** These are the features or attributes of the data point that the perceptron is trying to classify.
- **Weights ($w_1, w_2, \dots, w_n$):** Each input is multiplied by a corresponding weight. Weights represent the importance of each input feature. During training, these weights are adjusted.
- **Bias (b):** An additional constant input that shifts the activation function. It allows the decision boundary to be moved away from the origin.

2. Summation Function:

- The weighted sum of the inputs and the bias is calculated: $z = (x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n) + b$

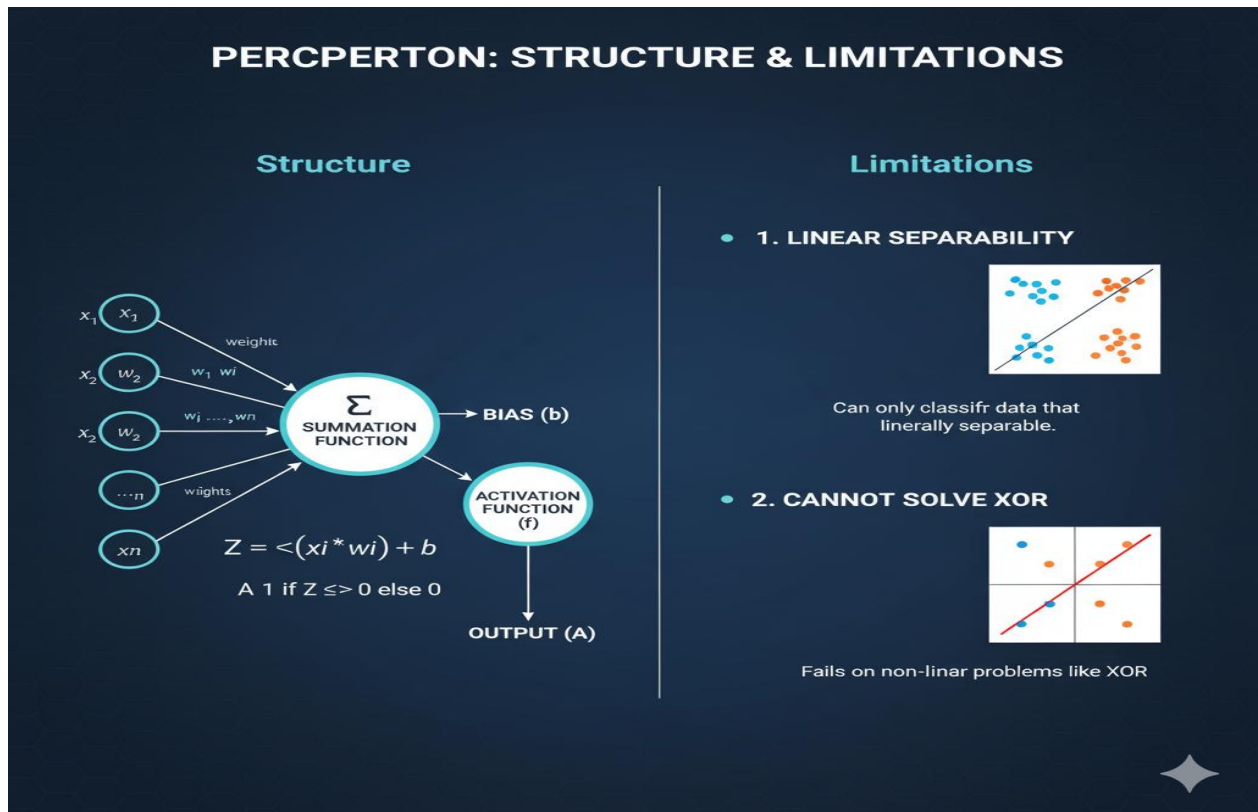
3. Activation Function:

- A step function (or threshold function) is typically used for a simple perceptron.
- If the weighted sum z exceeds a certain threshold (often 0), the perceptron outputs one value (e.g., 1 or "true").

- Otherwise, it outputs another value (e.g., 0 or "false").
- Commonly, the activation $A = f(Z)$ where $f(Z) = 1$ if $Z \geq 0$ and $f(Z) = 0$ if $Z < 0$.

4. Output Layer:

- **Output (A):** The final binary classification (e.g., 0 or 1).



4. Perceptron on AND Gate, OR Gate, XOR Gate

A single-layer perceptron can perfectly classify data that is **linearly separable**. This means you can draw a single straight line (or hyperplane in higher dimensions) to separate the different classes of data points. Let's see how this applies to logical gates.

We'll define the output for each gate:

- **Input (x1, x2):** Can be 0 or 1.
- **Output (y):** 0 or 1.

For a perceptron, the output is typically calculated as $y = \text{step}(w_1 * x_1 + w_2 * x_2 + b)$, where $\text{step}(z)$ is 1 if $z \geq 0$ and 0 if $z < 0$. Our goal is to find appropriate weights (w_1, w_2) and bias (b).

AND Gate

The AND gate outputs 1 only if *both* inputs are 1; otherwise, it outputs 0.

x1 x2 Output

0 0 0

0 1 0

1 0 0

1 1 1

[Export to Sheets](#)

Can a perceptron solve the AND gate? Yes, it can! We need to find weights and a bias such that only $(1, 1)$ results in a sum greater than or equal to 0, while all other combinations result in a sum less than 0.

Example Perceptron Parameters: Let $w_1 = 0.5$, $w_2 = 0.5$, $b = -0.7$

Let's check:

- **(0, 0):** $0.5*0 + 0.5*0 - 0.7 = -0.7$. Since $-0.7 < 0$, output is **0**. (Correct)
- **(0, 1):** $0.5*0 + 0.5*1 - 0.7 = -0.2$. Since $-0.2 < 0$, output is **0**. (Correct)
- **(1, 0):** $0.5*1 + 0.5*0 - 0.7 = -0.2$. Since $-0.2 < 0$, output is **0**. (Correct)
- **(1, 1):** $0.5*1 + 0.5*1 - 0.7 = 0.3$. Since $0.3 \geq 0$, output is **1**. (Correct)

The AND gate is linearly separable; you can draw a line to separate the $(1, 1)$ point from the other three points.

OR Gate

The OR gate outputs 1 if *at least one* input is 1; it outputs 0 only if both inputs are 0.

x1 x2 Output

0 0 0

0 1 1

1 0 1

1 1 1

[Export to Sheets](#)

Can a perceptron solve the OR gate? Yes, it can! We need weights and a bias such that only $(0, 0)$ results in a sum less than 0, and all other combinations result in a sum greater than or equal to 0.

Example Perceptron Parameters: Let $w_1 = 0.5$, $w_2 = 0.5$, $b = -0.2$

Let's check:

- **(0, 0):** $0.5*0 + 0.5*0 - 0.2 = -0.2$. Since $-0.2 < 0$, output is **0**. (Correct)

- **(0, 1):** $0.5*0 + 0.5*1 - 0.2 = 0.3$. Since $0.3 \geq 0$, output is **1**. (Correct)
- **(1, 0):** $0.5*1 + 0.5*0 - 0.2 = 0.3$. Since $0.3 \geq 0$, output is **1**. (Correct)
- **(1, 1):** $0.5*1 + 0.5*1 - 0.2 = 0.8$. Since $0.8 \geq 0$, output is **1**. (Correct)

The OR gate is also linearly separable; you can draw a line to separate the (0, 0) point from the other three points.

XOR Gate

The XOR (exclusive OR) gate outputs 1 if *exactly one* of the inputs is 1; if both inputs are the same (both 0 or both 1), it outputs 0.

x1 x2 Output

0 0 0

0 1 1

1 0 1

1 1 0

[Export to Sheets](#)

Can a perceptron solve the XOR gate? No, a *single-layer* perceptron cannot solve the XOR gate. This was a significant finding by Marvin Minsky and Seymour Papert in 1969, highlighting the limitations of early AI models.

Why? The XOR problem is **not linearly separable**. If you plot these points on a 2D graph:

- (0, 0) and (1, 1) are in one class (output 0).
- (0, 1) and (1, 0) are in another class (output 1).

There is no single straight line you can draw on a 2D plane that will perfectly separate the points where the output is 0 from the points where the output is 1.

To solve the XOR problem, you need a **multi-layer perceptron** (a neural network with at least one hidden layer). The hidden layer allows the network to learn non-linear decision boundaries by transforming the input data into a higher-dimensional space where it *becomes* linearly separable.

PERCEPTRON & LOGICAL GATES

AND Gate

AND Gate

Input	Output
(0,0)	0
(0,1)	0
(1,1)	1

SOLVABLE:
Linearly Separable

OR Gate

Input	Output
(0,0)	0
(0,1)	1
(1,0)	1
(1,1)	1

$0.5x_1 + 0.5x_2 - 0.7 = 0$

SOLVABLE:
Linearly Separable

Limitations

XOR Gate

Input	Output
(0,0)	0
(0,1)	1
(1,0)	0
(1,1)	1

$0.5x_1 + 0.5x_2 - 0.2 = 0$

NOT SOLVABLE by
Single Perceptron
Not Linearly Separable

XOR Gate

Input	Output
(0,0)	0
(0,1)	1
(1,0)	0
(1,1)	1

0 else 0

Requires a Multi-
Layer Perceptron (MLP).
Hidden layers learn
non-linear boundaries

Blue Dots: Output 0
Green Line: Decision Boundary

